Core BrainSurgery Primitives: Bulk Targeting, Slicing, and Assertion-Backed Validation

Before the MoE and PHLoRA case studies, the reader sees the smaller ideas that make those larger workflows possible.

A. Bulk Tensor Targeting

```
transforms:
  - cast:
      from: "model\.layers\.(\d+)\.self_attn\.
             (q_proj|k_proj|v_proj|o_proj)\.weight"
      to:   "attn_fp16.\g<0>"
      dtype: float16

# 16 layers x 4 projections = 64 tensors
# one declarative operation, no Python loop
```

Regex-based tensor references let one plan step apply to an entire tensor family at model scale.

B. Targeting with Slices

```
transforms:
  - copy:
      from: "model.layers.0.self_attn.
             q_proj.weight::[:128, :128]"
      to:   "debug.layer0.q_proj_block"

  - dump:
      target: "debug.layer0.q_proj_block"
      format: compact
```

The same reference syntax supports localized edits, inspection, and debugging on tensor subregions.

C. Verification as Code

```
inputs:
  - src::olmo_1b_0724_hf_dense
  - out::olmo_1b_0724_hf_dense_moe_demo

transforms:
  - assert: { exists: out::model.layers.0.mlp.
              experts.0.gate_proj.weight }
  - assert:
      shape: { of: out::model.layers.0.mlp.
               gate.weight, is: [2, 2048] }
  - assert:
      equal:
        left:  src::model.layers.0.mlp.
               gate_proj.weight::[:16, :16]
        right: out::model.layers.0.mlp.experts.0.
               gate_proj.weight::[:16, :16]
  - assert: { not: { exists: out::model.layers.0.
               mlp.gate_proj.weight } }
```

Expected postconditions become executable invariants, reducing silent checkpoint-editing failures.

Model-scale selection -> precise sub-tensor editing -> executable validation